Image Classification using MNIST image data (PyTorch)

```
# PyTorch
import torch
print(f"Is CUDA available? {torch.cuda.is_available()}")
print(f"GPU Name: {torch.cuda.get_device_name(0)}")
print(f"CUDA Version: {torch.version.cuda}")
import torchvision
import torchvision.transforms as transforms
import torch.nn as nn
import torch.nn.functional as nnFunc
import torch.amp as amp
# General
import numpy as np
import matplotlib.pyplot as plt
# Optimizer
import torch.optim as optim
# pretty print
from pprint import pprint
```

```
Is CUDA available? True
GPU Name: NVIDIA GeForce RTX 5050 Laptop GPU
CUDA Version: 13.0
```

Load image data

https://www.kaggle.com/datasets/hojjatk/mnist-dataset

1. Transform setting

The original images are in PIL (0 to 255 for each pixel). First we convert it to a tensor format (0 to 1). Then to make the data to be zero centering, we make the range to be between -1 to 1. If we use the following equation for each data, we can make it zero centered: \begin{equation} output = \frac{input - mean}{std} \end{equation} This is being done by setting the `mean` and `standard deviation` to be 0.5.

```
transform = transforms.Compose( [ transforms.RandomHorizontalFlip(),
                                  transforms.RandomCrop(28, padding=4),
                                  transforms.ToTensor(),
                                  transforms.Normalize((0.5,),
                                                       (0.5,))
                                 ])
```

2. Making the training set

Now we load the actual dataset and make the training dataset.

1. Load MNIST dataset (Download and prepare dataset)
    a. Put it in `./data_MNIST` directory.
    b. Download it
    c. Do the specified transformation on the dataset
2. Create the data loader (pipelining settings)
    a. Brake it to batches with size of 32
    b. Shuffle the deck in every epoch
    c. Use 4 number of workers to make the training faster

```
batchSize = 1024
numberOfWorkers = 0

trainSet = torchvision.datasets.MNIST(root='./data_MNIST',
                                      train=True,
                                      download=True,
                                      transform=transform)
trainLoader = torch.utils.data.DataLoader(trainSet,
                                          batch_size=batchSize,
                                          shuffle=True,
                                          num_workers=numberOfWorkers,
                                          pin_memory=True,
                                          persistent_workers=False)
```

3. Making the test set

Now making the test set. The procedure is the same as making the training set with some minor modifications.

```
testSet = torchvision.datasets.MNIST(root='./data_MNIST',
                                     train=False, # Diff
                                     download=True,
                                     transform=transform)
testLoader = torch.utils.data.DataLoader(testSet,
                                         batch_size=batchSize,
                                         shuffle=False, # Diff
                                         num_workers=numberOfWorkers,
                                         pin_memory=True,
                                         persistent_workers=False)

viz_batchSize = 4
vizLoader = torch.utils.data.DataLoader(testSet,
                                        batch_size=viz_batchSize,
                                        shuffle=True)
```

4. The classes

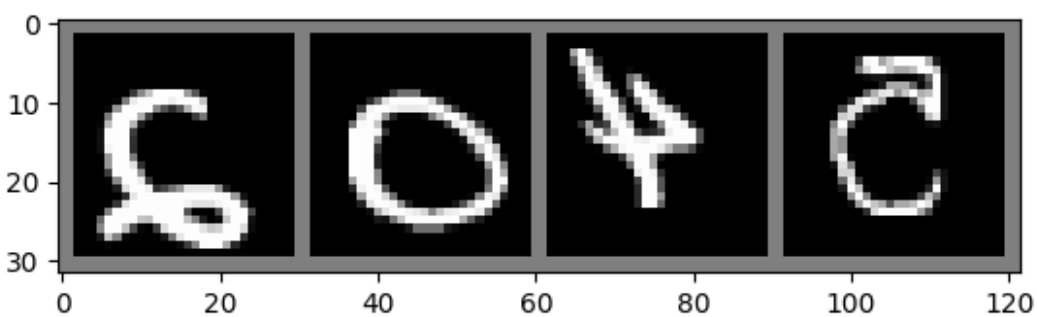
```
classes = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
print(classes)
```

```
['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

DEUBG1

Showing some of the images from the dataset.

```
# functions to show an image
def imshow(img):
  img = img / 2 + 0.5 # unnormalize
  npimg = img.numpy()
  plt.imshow(np.transpose(npimg, (1, 2, 0)))
  plt.show()
# get some random training images
dataiter = iter(vizLoader)
images, labels = next(dataiter)
# show images
imshow(torchvision.utils.make_grid(images))
# print labels
pprint(' '.join(f'{classes[labels[j]]:5s}' for j in
range(viz_batchSize)))
```




```
'2     0     4     5     '
```

5. Defining the CNN model

```
class Net(nn.Module):
  def __init__(self):
    super().__init__()
    self.conv1    = nn.Conv2d(1, 6, 5, padding='same')
    self.pool1    = nn.AvgPool2d(2, 2)
    self.conv2    = nn.Conv2d(6, 16, 5)
    self.pool2    = nn.AvgPool2d(2, 2)
```

```
    self.fc1      = nn.Linear(16 * 5 * 5, 120)
    self.fc2      = nn.Linear(120, 84)
    self.fc3      = nn.Linear(84, 10)

  def forward(self, x):
    x = self.pool1(torch.tanh(self.conv1(x)))
    x = self.pool2(torch.tanh(self.conv2(x)))
    x = x.view(-1, 16 * 5 * 5)
    x = nnFunc.relu(self.fc1(x))
    x = nnFunc.relu(self.fc2(x))
    x = self.fc3(x)

    return x
```

6. Instansiating the CNN model

```
def init_weights(m):
    if isinstance(m, nn.Conv2d) or isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, mode='fan_out',
nonlinearity='relu')
        if m.bias is not None:
            nn.init.constant_(m.bias, 0)
net = Net()
net.apply(init_weights)
```

```
Net(
  (conv1): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1),
padding=same)
  (pool1): AvgPool2d(kernel_size=2, stride=2, padding=0)
  (conv2): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
  (pool2): AvgPool2d(kernel_size=2, stride=2, padding=0)
  (fc1): Linear(in_features=400, out_features=120, bias=True)
  (fc2): Linear(in_features=120, out_features=84, bias=True)
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)
```

7. Move the model to GPU

```
device = torch.device('cuda:0' if torch.cuda.is_available() else
'cpu')
print(device)
```

```
cuda:0
```

```
net.to(device)
```

```
Net(
  (conv1): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1),
padding=same)
  (pool1): AvgPool2d(kernel_size=2, stride=2, padding=0)
```

```
  (conv2): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
  (pool2): AvgPool2d(kernel_size=2, stride=2, padding=0)
  (fc1): Linear(in_features=400, out_features=120, bias=True)
  (fc2): Linear(in_features=120, out_features=84, bias=True)
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)
```

7. Degine loss function and optimizer

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
mode='max', factor=0.1, patience=10)
```

8. Traing the network

```
scaler = amp.GradScaler('cuda')

epochs = 30
printFrequency = len(trainLoader) // 4

for epoch in range(epochs):
    net.train()
    running_loss = 0.0
    correct = 0
    total = 0

    # --- TRAINING PHASE ---
    for i, data in enumerate(trainLoader, 0):
        inputs, labels = data[0].to(device, non_blocking=True),
data[1].to(device, non_blocking=True)

        optimizer.zero_grad()

        with amp.autocast('cuda'):
            outputs = net(inputs)
            loss = criterion(outputs, labels)

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(net.parameters(), max_norm=1.0)

        scaler.step(optimizer)
        scaler.update()

        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
        running_loss += loss.item()
```

```
        if i % printFrequency == (printFrequency - 1):
            accuracy = 100 * correct / total
            print(f'Epoch {epoch + 1} | Batch {i +
1}/{len(trainLoader)} | loss: {running_loss / printFrequency:.3f} |
acc: {accuracy:.2f}%')
            running_loss, correct, total = 0.0, 0, 0

    # --- VALIDATION PHASE ---
    net.eval()
    val_correct = 0
    val_total = 0
    with torch.no_grad():
        for data in testLoader:
            images, labels = data[0].to(device), data[1].to(device)
            with amp.autocast('cuda'):
                outputs = net(images)
            _, predicted = torch.max(outputs.data, 1)
            val_total += labels.size(0)
            val_correct += (predicted == labels).sum().item()

    val_acc = 100 * val_correct / val_total
    current_lr = optimizer.param_groups[0]['lr']
    print(f'>> Epoch {epoch + 1} SUMMARY: Val Acc: {val_acc:.2f}% |
Current LR: {current_lr}')

    # --- AUTOMATIC LR DROP ---
    scheduler.step(val_acc)

print('Finished Training')
```

```
Epoch 1 | Batch 14/59 | loss: 2.672 | acc: 13.24%
Epoch 1 | Batch 28/59 | loss: 2.023 | acc: 29.47%
Epoch 1 | Batch 42/59 | loss: 1.723 | acc: 40.44%
Epoch 1 | Batch 56/59 | loss: 1.464 | acc: 50.60%
>> Epoch 1 SUMMARY: Val Acc: 56.64% | Current LR: 0.001
Epoch 2 | Batch 14/59 | loss: 1.212 | acc: 59.56%
Epoch 2 | Batch 28/59 | loss: 1.034 | acc: 66.41%
Epoch 2 | Batch 42/59 | loss: 0.915 | acc: 70.21%
Epoch 2 | Batch 56/59 | loss: 0.831 | acc: 73.28%
>> Epoch 2 SUMMARY: Val Acc: 75.49% | Current LR: 0.001
Epoch 3 | Batch 14/59 | loss: 0.752 | acc: 75.92%
Epoch 3 | Batch 28/59 | loss: 0.713 | acc: 77.40%
Epoch 3 | Batch 42/59 | loss: 0.672 | acc: 78.38%
Epoch 3 | Batch 56/59 | loss: 0.640 | acc: 79.53%
>> Epoch 3 SUMMARY: Val Acc: 80.67% | Current LR: 0.001
Epoch 4 | Batch 14/59 | loss: 0.602 | acc: 80.78%
Epoch 4 | Batch 28/59 | loss: 0.590 | acc: 80.97%
Epoch 4 | Batch 42/59 | loss: 0.561 | acc: 81.76%
Epoch 4 | Batch 56/59 | loss: 0.550 | acc: 82.42%
```

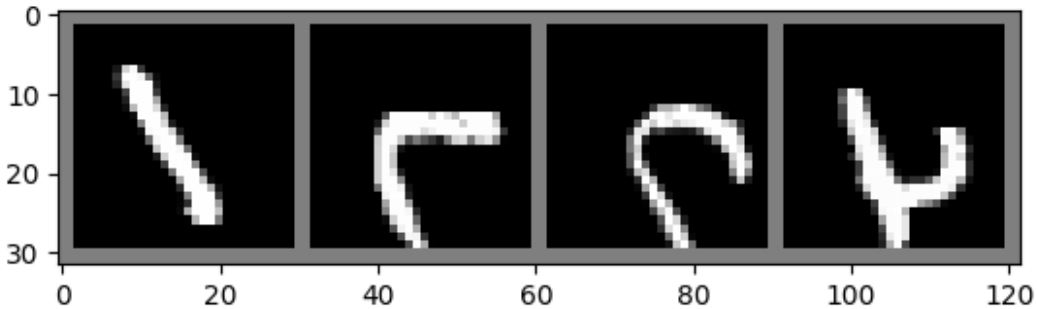
```
>> Epoch 4 SUMMARY: Val Acc: 83.60% | Current LR: 0.001
Epoch 5 | Batch 14/59 | loss: 0.528 | acc: 83.28%
Epoch 5 | Batch 28/59 | loss: 0.517 | acc: 83.87%
Epoch 5 | Batch 42/59 | loss: 0.508 | acc: 83.11%
Epoch 5 | Batch 56/59 | loss: 0.475 | acc: 85.02%
>> Epoch 5 SUMMARY: Val Acc: 85.76% | Current LR: 0.001
Epoch 6 | Batch 14/59 | loss: 0.475 | acc: 84.72%
Epoch 6 | Batch 28/59 | loss: 0.465 | acc: 85.20%
Epoch 6 | Batch 42/59 | loss: 0.443 | acc: 85.85%
Epoch 6 | Batch 56/59 | loss: 0.439 | acc: 85.75%
>> Epoch 6 SUMMARY: Val Acc: 85.93% | Current LR: 0.001
Epoch 7 | Batch 14/59 | loss: 0.420 | acc: 86.36%
Epoch 7 | Batch 28/59 | loss: 0.430 | acc: 86.17%
Epoch 7 | Batch 42/59 | loss: 0.418 | acc: 86.89%
Epoch 7 | Batch 56/59 | loss: 0.409 | acc: 86.72%
>> Epoch 7 SUMMARY: Val Acc: 87.81% | Current LR: 0.001
Epoch 8 | Batch 14/59 | loss: 0.398 | acc: 87.30%
Epoch 8 | Batch 28/59 | loss: 0.405 | acc: 87.12%
Epoch 8 | Batch 42/59 | loss: 0.390 | acc: 87.59%
Epoch 8 | Batch 56/59 | loss: 0.390 | acc: 87.51%
>> Epoch 8 SUMMARY: Val Acc: 87.77% | Current LR: 0.001
Epoch 9 | Batch 14/59 | loss: 0.388 | acc: 87.63%
Epoch 9 | Batch 28/59 | loss: 0.380 | acc: 87.93%
Epoch 9 | Batch 42/59 | loss: 0.383 | acc: 87.94%
Epoch 9 | Batch 56/59 | loss: 0.366 | acc: 88.66%
>> Epoch 9 SUMMARY: Val Acc: 88.73% | Current LR: 0.001
Epoch 10 | Batch 14/59 | loss: 0.372 | acc: 88.28%
Epoch 10 | Batch 28/59 | loss: 0.365 | acc: 88.43%
Epoch 10 | Batch 42/59 | loss: 0.341 | acc: 89.21%
Epoch 10 | Batch 56/59 | loss: 0.353 | acc: 88.69%
>> Epoch 10 SUMMARY: Val Acc: 89.19% | Current LR: 0.001
Epoch 11 | Batch 14/59 | loss: 0.348 | acc: 88.90%
Epoch 11 | Batch 28/59 | loss: 0.352 | acc: 88.76%
Epoch 11 | Batch 42/59 | loss: 0.344 | acc: 89.38%
Epoch 11 | Batch 56/59 | loss: 0.341 | acc: 89.03%
>> Epoch 11 SUMMARY: Val Acc: 89.65% | Current LR: 0.001
Epoch 12 | Batch 14/59 | loss: 0.328 | acc: 89.68%
Epoch 12 | Batch 28/59 | loss: 0.322 | acc: 89.60%
Epoch 12 | Batch 42/59 | loss: 0.326 | acc: 89.32%
Epoch 12 | Batch 56/59 | loss: 0.328 | acc: 89.86%
>> Epoch 12 SUMMARY: Val Acc: 89.53% | Current LR: 0.001
Epoch 13 | Batch 14/59 | loss: 0.317 | acc: 89.89%
Epoch 13 | Batch 28/59 | loss: 0.326 | acc: 89.71%
Epoch 13 | Batch 42/59 | loss: 0.325 | acc: 89.67%
Epoch 13 | Batch 56/59 | loss: 0.320 | acc: 89.73%
>> Epoch 13 SUMMARY: Val Acc: 90.63% | Current LR: 0.001
Epoch 14 | Batch 14/59 | loss: 0.309 | acc: 90.10%
Epoch 14 | Batch 28/59 | loss: 0.306 | acc: 90.39%
Epoch 14 | Batch 42/59 | loss: 0.292 | acc: 90.68%
```

```
Epoch 14 | Batch 56/59 | loss: 0.308 | acc: 90.76%
>> Epoch 14 SUMMARY: Val Acc: 90.81% | Current LR: 0.001
Epoch 15 | Batch 14/59 | loss: 0.289 | acc: 90.62%
Epoch 15 | Batch 28/59 | loss: 0.297 | acc: 90.33%
Epoch 15 | Batch 42/59 | loss: 0.299 | acc: 90.58%
Epoch 15 | Batch 56/59 | loss: 0.293 | acc: 90.81%
>> Epoch 15 SUMMARY: Val Acc: 91.13% | Current LR: 0.001
Epoch 16 | Batch 14/59 | loss: 0.294 | acc: 90.49%
Epoch 16 | Batch 28/59 | loss: 0.297 | acc: 90.60%
Epoch 16 | Batch 42/59 | loss: 0.280 | acc: 91.06%
Epoch 16 | Batch 56/59 | loss: 0.284 | acc: 90.89%
>> Epoch 16 SUMMARY: Val Acc: 91.46% | Current LR: 0.001
Epoch 17 | Batch 14/59 | loss: 0.271 | acc: 91.57%
Epoch 17 | Batch 28/59 | loss: 0.281 | acc: 91.05%
Epoch 17 | Batch 42/59 | loss: 0.275 | acc: 91.33%
Epoch 17 | Batch 56/59 | loss: 0.293 | acc: 90.65%
>> Epoch 17 SUMMARY: Val Acc: 91.38% | Current LR: 0.001
Epoch 18 | Batch 14/59 | loss: 0.277 | acc: 91.28%
Epoch 18 | Batch 28/59 | loss: 0.279 | acc: 91.22%
Epoch 18 | Batch 42/59 | loss: 0.264 | acc: 91.68%
Epoch 18 | Batch 56/59 | loss: 0.276 | acc: 91.26%
>> Epoch 18 SUMMARY: Val Acc: 91.49% | Current LR: 0.001
Epoch 19 | Batch 14/59 | loss: 0.265 | acc: 91.78%
Epoch 19 | Batch 28/59 | loss: 0.281 | acc: 91.39%
Epoch 19 | Batch 42/59 | loss: 0.261 | acc: 91.57%
Epoch 19 | Batch 56/59 | loss: 0.260 | acc: 91.82%
>> Epoch 19 SUMMARY: Val Acc: 91.46% | Current LR: 0.001
Epoch 20 | Batch 14/59 | loss: 0.246 | acc: 92.15%
Epoch 20 | Batch 28/59 | loss: 0.252 | acc: 91.95%
Epoch 20 | Batch 42/59 | loss: 0.255 | acc: 91.97%
Epoch 20 | Batch 56/59 | loss: 0.270 | acc: 91.42%
>> Epoch 20 SUMMARY: Val Acc: 92.32% | Current LR: 0.001
Epoch 21 | Batch 14/59 | loss: 0.256 | acc: 91.87%
Epoch 21 | Batch 28/59 | loss: 0.248 | acc: 92.27%
Epoch 21 | Batch 42/59 | loss: 0.256 | acc: 91.80%
Epoch 21 | Batch 56/59 | loss: 0.250 | acc: 92.01%
>> Epoch 21 SUMMARY: Val Acc: 92.68% | Current LR: 0.001
Epoch 22 | Batch 14/59 | loss: 0.254 | acc: 91.88%
Epoch 22 | Batch 28/59 | loss: 0.234 | acc: 92.47%
Epoch 22 | Batch 42/59 | loss: 0.245 | acc: 92.08%
Epoch 22 | Batch 56/59 | loss: 0.237 | acc: 92.52%
>> Epoch 22 SUMMARY: Val Acc: 92.94% | Current LR: 0.001
Epoch 23 | Batch 14/59 | loss: 0.234 | acc: 92.35%
Epoch 23 | Batch 28/59 | loss: 0.246 | acc: 92.45%
Epoch 23 | Batch 42/59 | loss: 0.234 | acc: 92.66%
Epoch 23 | Batch 56/59 | loss: 0.247 | acc: 92.44%
>> Epoch 23 SUMMARY: Val Acc: 92.19% | Current LR: 0.001
Epoch 24 | Batch 14/59 | loss: 0.245 | acc: 92.24%
Epoch 24 | Batch 28/59 | loss: 0.234 | acc: 93.00%
```

```
Epoch 24 | Batch 42/59 | loss: 0.228 | acc: 92.65%
Epoch 24 | Batch 56/59 | loss: 0.233 | acc: 92.62%
>> Epoch 24 SUMMARY: Val Acc: 93.07% | Current LR: 0.001
Epoch 25 | Batch 14/59 | loss: 0.225 | acc: 92.97%
Epoch 25 | Batch 28/59 | loss: 0.242 | acc: 92.54%
Epoch 25 | Batch 42/59 | loss: 0.228 | acc: 92.66%
Epoch 25 | Batch 56/59 | loss: 0.226 | acc: 92.76%
>> Epoch 25 SUMMARY: Val Acc: 92.86% | Current LR: 0.001
Epoch 26 | Batch 14/59 | loss: 0.224 | acc: 93.09%
Epoch 26 | Batch 28/59 | loss: 0.217 | acc: 93.14%
Epoch 26 | Batch 42/59 | loss: 0.227 | acc: 92.95%
Epoch 26 | Batch 56/59 | loss: 0.223 | acc: 93.00%
>> Epoch 26 SUMMARY: Val Acc: 92.91% | Current LR: 0.001
Epoch 27 | Batch 14/59 | loss: 0.215 | acc: 93.36%
Epoch 27 | Batch 28/59 | loss: 0.222 | acc: 93.04%
Epoch 27 | Batch 42/59 | loss: 0.220 | acc: 93.10%
Epoch 27 | Batch 56/59 | loss: 0.216 | acc: 92.97%
>> Epoch 27 SUMMARY: Val Acc: 93.43% | Current LR: 0.001
Epoch 28 | Batch 14/59 | loss: 0.221 | acc: 93.06%
Epoch 28 | Batch 28/59 | loss: 0.220 | acc: 93.15%
Epoch 28 | Batch 42/59 | loss: 0.214 | acc: 93.15%
Epoch 28 | Batch 56/59 | loss: 0.215 | acc: 93.23%
>> Epoch 28 SUMMARY: Val Acc: 93.01% | Current LR: 0.001
Epoch 29 | Batch 14/59 | loss: 0.211 | acc: 93.14%
Epoch 29 | Batch 28/59 | loss: 0.222 | acc: 92.87%
Epoch 29 | Batch 42/59 | loss: 0.213 | acc: 93.32%
Epoch 29 | Batch 56/59 | loss: 0.207 | acc: 93.53%
>> Epoch 29 SUMMARY: Val Acc: 93.39% | Current LR: 0.001
Epoch 30 | Batch 14/59 | loss: 0.211 | acc: 93.52%
Epoch 30 | Batch 28/59 | loss: 0.212 | acc: 93.24%
Epoch 30 | Batch 42/59 | loss: 0.209 | acc: 93.28%
Epoch 30 | Batch 56/59 | loss: 0.199 | acc: 93.62%
>> Epoch 30 SUMMARY: Val Acc: 93.59% | Current LR: 0.001
Finished Training
```

9. Test the network on the test data

```
dataIter = iter(vizLoader)
images, labels = next(dataIter)
# print images
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join(f'{classes[labels[j]]:5s}' for j in
range(viz_batchSize)))
```




```
GroundTruth:  1     7     7     4
```

```
# Move the data to GPU
images, labels = images.to(device), labels.to(device)

# Now you can run it
outputs = net(images)
_, predicted = torch.max(outputs, 1)

# IMPORTANT: To print, move back to CPU
# Printing needs the labels to be on the CPU so Python can read them
print('Predicted: ', ' '.join(f'{classes[predicted[j].item()]:5s}' for
j in range(len(predicted))))
```

```
Predicted:  1     7     7     4
```

10. Test the network on the whole dataset

```
test_loss = 0.0
correct = 0
total = 0

with torch.no_grad():
    for data in testLoader:
        images, labels = data[0].to(device), data[1].to(device)

        outputs = net(images)

        # 1. Calculate the loss for this batch
        loss = criterion(outputs, labels)
        test_loss += loss.item()

        # 2. Calculate accuracy as before
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

# Calculate the average loss over all batches
avg_test_loss = test_loss / len(testLoader)
```

```
print(f'Test Loss: {avg_test_loss:.4f}')
print(f'Test Accuracy: {100 * correct / total:.2f}%')
```

```
Test Loss: 0.2019
Test Accuracy: 93.74%
```

11. Check performance within each class

```
# prepare to count predictions for each class
correct_pred = {classname: 0 for classname in classes}
total_pred = {classname: 0 for classname in classes}

# again no gradients needed
with torch.no_grad():
    for data in testLoader:
        images, labels = data[0].to(device), data[1].to(device)
        outputs = net(images)
        _, predictions = torch.max(outputs, 1)
        # collect the correct predictions for each class
        for label, prediction in zip(labels, predictions):
            if label == prediction:
                correct_pred[classes[label]] += 1
            total_pred[classes[label]] += 1

# print accuracy for each class
for classname, correct_count in correct_pred.items():
    accuracy = 100 * float(correct_count) / total_pred[classname]
    print(f'Accuracy for class: {classname:5s} is {accuracy:.1f} %')
```

```
Accuracy for class: 0     is 96.5 %
Accuracy for class: 1     is 98.2 %
Accuracy for class: 2     is 88.9 %
Accuracy for class: 3     is 92.0 %
Accuracy for class: 4     is 96.0 %
Accuracy for class: 5     is 85.3 %
Accuracy for class: 6     is 97.4 %
Accuracy for class: 7     is 96.9 %
Accuracy for class: 8     is 96.1 %
Accuracy for class: 9     is 92.4 %
```